

【水仙花数】

题目描述

所谓 **水仙花数** ，是指一个n位的正整数，其各位数字的n次方和等于该数本身。

例如153是水仙花数，153是一个3位数，并且 $153 = 1^3 + 5^3 + 3^3$ 。

输入描述

第一行输入一个整数n，表示一个n位的正整数。n在3到7之间，包含3和7。

第二行输入一个整数m，表示需要返回第m个水仙花数。

输出描述

返回长度是n的第m个水仙花数。个数从0开始编号。

若m大于水仙花数的个数，返回最后一个水仙花数和m的乘积。

若输入不合法，返回-1。

用例

输入	3 0
输出	153
说明	153是第一个水仙花数

输入	9 1
输出	-1
说明	9超出范围

题目解析

这道题实现逻辑并不难，大家可以看下面算法源码。

但是本题的水仙花数最长可以有7位，也就是百万级别的，虽然下面算法差不多是 $O(n)$ 的，但是估计也Hold不住。

暂时还没想到更好的办法。

JS算法源码

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  const lines = [];
10 rl.on("line", (line) => {
11   lines.push(line);
12
13   if (lines.length === 2) {
14     let [n, m] = lines.map(Number);
15     console.log(getResult(n, m));
16     lines.length = 0;
17   }
18 });
```

运行

```
19 |
20 | function getResult(n, m) {
21 |     // 若输入不合法, 返回-1
22 |     if (n < 3 || n > 7 || m < 0) return -1;
23 |
24 |     // 提前计算好0~9的N次方, 避免后续进行重复计算
25 |     const powN = {};
26 |     for (let i = 0; i <= 9; i++) {
27 |         powN[i] = Math.pow(i, n);
28 |     }
29 |
30 |     // N位数的最小值
31 |     const minNum = Math.pow(10, n - 1);
32 |     // N位数的最大值 + 1
33 |     const maxNum = Math.pow(10, n);
34 |
35 |     // 记录当前水仙花数
36 |     let ans = 0;
37 |
38 |     // 记录当前水仙花数的索引
39 |     let idx = 0;
40 |
41 |     // 遍历所有的N位数
42 |     for (let num = minNum; num < maxNum; num++) {
43 |         // 记录num各位数字的N次方之和
44 |         let sum = 0;
45 |
46 |         // 遍历num的每一位数字
47 |         for(let c of (num + '')) {
48 |             sum += powN[c];
49 |         }
50 |
51 |         // 判断num是否为水仙花数
52 |         if (num == sum) {
53 |             ans = num;
54 |             // 如果num刚好是N位数的第m个水仙花数, 则直接返回, 否则继续查找
55 |             if (idx++ == m) return ans;
```

```

56     }57 |   }
58
59     // 若 $m$ 大于水仙花数的个数，返回最后一个水仙花数和 $m$ 的乘积
60     return ans * m;
61 }

```

Java算法源码

```

1  import java.util.Arrays;
2  import java.util.HashMap;
3  import java.util.Scanner;
4
5  public class Main {
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8
9          int n = sc.nextInt();
10         int m = sc.nextInt();
11
12         System.out.println(getResult(n, m));
13     }
14
15     public static long getResult(int n, int m) {
16         // 若输入不合法，返回-1
17         if (n < 3 || n > 7 || m < 0) return -1;
18
19         // 提前计算好0~9的 $N$ 次方，避免后续进行重复计算
20         HashMap<Character, Integer> powN = new HashMap<>();
21         for (int i = 0; i <= 9; i++) {
22             // 将整型0~9转化字符'0'~'9'，即让 $i + '0'$ 即可
23             powN.put((char) (i + '0'), (int) Math.pow(i, n));
24         }
25

```

```

26 // 最小的N位数27 |      int min = (int) Math.pow(10, n - 1);
28 // 最大的N位数
29 int max = (int) Math.pow(10, n);
30
31 // 记录当前水仙花数
32 long ans = 0;
33
34 // 记录当前水仙花数是第几个
35 int idx = 0;
36
37 for (int num = min; num < max; num++) {
38     // 记录num各位数字的N次方之和
39     int sum = 0;
40
41     // 遍历num的每一位数字
42     String str = num + "";
43     for(int i=0; i<n; i++) {
44         sum += powN.get(str.charAt(i));
45     }
46
47     // 判断num是否为水仙花数
48     if (sum == num) {
49         ans = num;
50         // 如果num刚好是N位数的第m个水仙花数，则直接返回，否则继续查找
51         if (idx++ == m) return ans;
52     }
53 }
54
55 // 若m大于水仙花数的个数，返回最后一个水仙花数和m的乘积
56 return ans * m;
57 }
58 }

```

```
1  import math
2
3  # 输入获取
4  n = int(input())
5  m = int(input())
6
7
8  # 算法入口
9  def getResult():
10     # 若输入不合法, 返回-1
11     if n < 3 or n > 7 or m < 0:
12         return -1
13
14     # 提前计算好0~9的N次方, 避免后续进行重复计算
15     powN = {}
16     for i in range(10):
17         powN[str(i)] = int(math.pow(i, n))
18
19     # 最小的N位数
20     minV = int(math.pow(10, n-1))
21     # 最大的N位数
22     maxV = int(math.pow(10, n))
23
24     # 记录当前水仙花数
25     ans = 0
26
27     # 记录当前水仙花数是第几个
28     idx = 0
29
30     for num in range(minV, maxV):
31         # 记录num各位数字的N次方之和
32         sumV = 0
33
34         # 遍历num的每一位数字
35         for c in str(num):
36             sumV += powN[c]
```

```

37 |
38 |         # 判断num是否为水仙花数
39 |         if sumV == num:
40 |             ans = num
41 |
42 |         # 如果num刚好是N位数的第m个水仙花数，则直接返回，否则继续查找
43 |         if idx == m:
44 |             return ans
45 |
46 |         idx += 1
47 |
48 |     # 若m大于水仙花数的个数，返回最后一个水仙花数和m的乘积
49 |     return ans * m
50 |
51 |
52 | # 算法调用
53 | print(getResult())

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <math.h>
3 |
4 | long getResult(int n, int m);
5 |
6 | int main() {
7 |     int n, m;
8 |     scanf("%d %d", &n, &m);
9 |
10 |    printf("%ld\n", getResult(n, m));
11 |
12 |    return 0;
13 | }
14 |
15 | long getResult(int n, int m) {

```

```
16 // 若输入不合法, 返回-1 | if(n < 3 || n > 7 || m < 0) return -1;
17
18
19 // 提前计算好0~9的N次方, 避免后续进行重复计算
20 int powN[10];
21 for(int i=0; i<10; i++) {
22     powN[i] = (int) pow(i, n);
23 }
24
25 // 最小的N位数
26 int min = (int) pow(10, n - 1);
27 // 最大的N位数
28 int max = (int) pow(10, n);
29
30 // 记录当前水仙花数
31 long ans = 0;
32
33 // 记录当前水仙花数是第几个
34 int idx = 0;
35
36 for(int num = min; num < max; num++) {
37     // 记录num各位数字的N次方之和
38     int sum = 0;
39
40     // 遍历num的每一位数字
41     int num_cp = num;
42     while(num_cp > 0) {
43         sum += powN[num_cp % 10];
44         num_cp /= 10;
45     }
46
47     // 判断num是否为水仙花数
48     if(sum == num) {
49         ans = num;
50         // 如果num刚好是N位数的第m个水仙花数, 则直接返回, 否则继续查找
51         if(idx++ == m) return ans;
52     }
53 }
```



```
54 |  
55 |         // 若 $m$ 大于水仙花数的个数，返回最后一个水仙花数和 $m$ 的乘积  
56 |         return ans * m;  
57 |     }
```

C++算法源码

```
1  #include <bits/stdc++.h>  
2  
3  using namespace std;  
4  
5  long solution(int n, int m) {  
6      // 若输入不合法，返回-1  
7      if (n < 3 || n > 7 || m < 0) return -1;  
8  
9      // 提前计算好0~9的 $N$ 次方，避免后续进行重复计算  
10     int powN[10];  
11     for (int i = 0; i < 10; i++) {  
12         powN[i] = (int) pow(i, n);  
13     }  
14  
15     // 最小的 $N$ 位数  
16     int low = (int) pow(10, n - 1);  
17     // 最大的 $N$ 位数  
18     int high = (int) pow(10, n);  
19  
20     // 记录当前水仙花数  
21     long ans = 0;  
22  
23     // 记录当前水仙花数是第几个  
24     int idx = 0;  
25  
26     for (int num = low; num < high; num++) {  
27         // 记录 $num$ 各位数字的 $N$ 次方之和  
28         int sum = 0;
```

```
29 |
30 |         // 遍历num的每一位数字
31 |         int num_cp = num;
32 |         while (num_cp > 0) {
33 |             sum += powN[num_cp % 10];
34 |             num_cp /= 10;
35 |         }
36 |
37 |         // 判断num是否为水仙花数
38 |         if (sum == num) {
39 |             ans = num;
40 |             // 如果num刚好是N位数的第m个水仙花数，则直接返回，否则继续查找
41 |             if (idx++ == m) return ans;
42 |         }
43 |     }
44 |
45 |     // 若m大于水仙花数的个数，返回最后一个水仙花数和m的乘积
46 |     return ans * m;
47 | }
48 |
49 | int main() {
50 |     int n, m;
51 |     cin >> n >> m;
52 |
53 |     cout << solution(n, m) << endl;
54 |
55 |     return 0;
56 | }
```